Code Challenge

Loops identify divisble integers

Loops can be used to identify if an integer is or is not evenly divisible by another integer. By the end of the loop, if the dividend is 0, then we know that there are no leftovers and it is evenly divisible by the divisor.

```
int divisor = someNumber;
int dividend = anotherNumber;

while (dividend >= divisor) {
  dividend -= divisor;
}

if (dividend == 0) {
  // then the dividend is divisible by the divisor
}
```

Loops compute sums

Loops can be used to compute a sum. Before we iterate through each element, then we just need to have a number to keep track of the current sum. Then on each iteration through the array, we can add that element to the sum. When the loop is finished, then we have the final sum.

```
// We can use a loop to add 1 to the sum 10 times
int sum = 0;
int addToSum = 1;

for (int i = 0; i < 10; i++) {
  sum += addToSum;
}
```

Loops identify digits in a number

Loops can be used to identify digits in a number. We can go through each digit in a number by taking mod 10 of the number. This will give us the remainder, which is effectively the ones digit of the number. Removing the ones digit from a number can be done by dividing the number by 10. We can repeat these steps using a loop until there are no more digits left in the number.

```
int number = 123;
while (number > 0) {
  int digit = number % 10;
  number /= 10; // take the ones digit off the number
}
```

Loops determine a minimum or maximum value.

Loops can be used to determine a minimum or maximum value. We just need to have a temporary variable to track the max number so far. On each iteration, we can compare the current element to the max number, and if the current element is larger, then we should replace the max number with it.

```
int[] listOfNumbers = {1, 1, 5, 3};

int maxNumber = listOfNumbers[0];
for (int i = 0; i < listOfNumbers.length;
i++) {
  if (listOfNumbers[i] > maxDigit) {
    maxDigit = listOfNumbers[i];
  }
}
```

Loops determine the frequency with which a specific criterion is met

Loops can be used to determine the frequency with which a specific criterion is met. We will need a counter to track the number of elements that satisfy a particular condition. Then we can wrap a condition with a for loop to make sure it is applied to all elements in the array. If the condition holds true, we can add one to the counter.

```
int[] numList = {1, 2, 3, 1};

int numOfNumsGreaterThanOne = 0;

for (int i = 0; i < numList.length; i++) {
  if (numList[i] > 1) {
    numOfNumsGreaterThanOne++;
  }
}
```

Loops can determine if at least one element has a particular property

Loops can determine if at least one element has a particular property

```
int[] numList = {1, 2, 3, 5};

boolean hasEvenNum = false;
for (int i = 0; i < numList.length; i++) {
  if (numList[i] % 2 == 0) {
    hasEvenNum = true;
  }
}
```

Loops can determine if all elements have a particular property

Loops can determine if all elements have a particular property. One way is to set up a condition to evaluate if one element of the list has the property. Then we need a boolean that is true as long as the condition holds true, and wrap a loop around the condition to check every element in the list.

```
int[] numList = {2, 4, 6, 8, 4};

int allEven = false;
for (int i = 0; i < numList.length; i++) {
  if (numList[i] % 2 == 1) {
    allEven = false;
  }
}
```

Loops can determine the presence or absence of duplicate elements

Loops can determine the presence or absence of duplicate elements. One way is to set up a loop inside another loop. In each iteration of the outer loop, we check if any of the elements in the inner loop are equal.

```
int[] numList = {1, 2, 3, 4, 4};

boolean hasDupe = false;
for (int i = 0; i < numList.length; i++) {
  for (int j = i; j < numList.length; j++) {
    if (i != j && numList[i] == numList[j]) {
      hasDupe = true;
    }
  }
}
```

Loops can be used to reverse the order of the elements

Loops can be used to reverse the order of the elements. One way is to set up a loop that goes through half of the elements of the array with two counters, one iterating forwards through the front of the array and the other iterating backwards from the end. Then we can just swap the elements at each counter (the first element with the last, the second with the second to last, and so on).

```
int back = list.length - 1;
for (int front = 0; i < list.length/2;
front++) {
  int temp = list[front];
  list[front] = list[back];
  list[back] = temp;
  back--;
}
```

Loops can be used to shift or rotate elements left or right.

Loops can be used to shift or rotate elements left or right. To rotate the elements, we set the last element in the array to an element that track the previous element in list. At the start, the previous element should be the last element in the list. Then we iterate through each element in the list and swap out the previous element with the current element.

```
int previous = list[list.length - 1];
for (int i = 0; i < list.length; i++) {
  int temp = list[i];
  list[i] = previous;
  previous = temp;
}
```

Loops can be used to access the elements in a list

Loops can be used to access all elements in a list. This is helpful when applying the same operation on every element in the list.

```
// Print out "Hello" 10 times
ArrayList<String> list = new
ArrayList<String>();
for (int i = 0; i < 10; i++) {
  list.add("Hello");
}

for (String s : list) {
  System.out.println(s);
}
```

ArrayLists can be used to apply the same standards to lists as regular arrays

ArrayLists can loop through arrays and update the elements similar to arrays.

```
ArrayList<Integer> arrayList = new
ArrayList<Integer>();
int[] intArray = new int[10];

for (int i = 0 ; i < 10; i++) {
  arrayList.add(i);
  intArray[i] = i;
}
```

Arrays can be used to add elements in ArrayLists

Loops can be used to add and remove elements in ArrayLists.

```
// add 10 strings in the list
ArrayList<String> list = new
ArrayList<String>();
for (int i = 0; i < 10; i++) {
  list.add("new string");
}
```

Some algorithms require multiple String, array, or ArrayList objects to be traversed simultaneously.

If two lists need to be traversed simultaneously, we can use the same counter to iterate through both of them, or two counters for each list. If one counter is used, we just need to account for the length of the arrays to make sure that there aren't out of bounds errors.

```
int firstListLength = firstList.size();
int secondListLength = secondList.size();
int numElementsToTraverse = 0;

if (firstListLength < secondListLength) {
  numElementsToTraverse = firstListLength;
} else {
  numElementsToTraverse = secondListLength;
}

for (int i = 0; i < numElementsToTraverse;
i++) {
  System.out.println(firstList.get(i));
  System.out.println(secondList.get(i));
}
```

String Traversal for Checking Substrings

Standard algorithms involving `String` traversal can be used to perform tasks such as finding if one (or more) substrings contain a certain property or determining the number of substrings that meet specific criteria.

```
public static int findNumValues(String text, String findText) {
  int count = 0;
  // iterate through String
  for (int i = 0; i <= text.length()-findText.length(); i++){
    // check if substring matches a criteria
    if (text.substring(i,i+findText.length()).equals(findText))     {
      // increase count
      count += 1
    }
  }
  return count;
}

public static void main(String[] args) {
  String text = "hello hi howdy hi goodbye";
  System.out.println(findNumValues(text, "hi"));
  // Prints: 2
}
```

String Traversal: Reversing a String

`String` traversal can be used to iterate through a `String` and create a new `String` with the characters reversed.

```
public static String reverseString(String text) {
  String reversed = new String("");
  for (int i=0; i < text.length(); i++){
    char nextCharacter = text.charAt(i);
    reversed = nextCharacter + reversed;
  }
  return reversed;
}

public static void main(String[] args) {
  String text = new String("greetings earthling");
  System.out.println(reverseString(text));
  // Prints: gnilhtrae sgniteerg
}
```

Iterating 2D Arrays

In order to access each element of a 2D array, use iteration to traverse through each row of the 2D array. Then, within each row, use a nested loop to traverse through each element of the inner array.

```
// Prints the total value of each element in 2D array

int[][] arr2D = {{17, 13, 19, 22}, {12, 18, 25, 20}, {15, 18, 21, 24}, {19, 23, 23, 22}, {18, 20, 21, 26}};

int total = 0;
for (int row = 0; row < arr2D.length; row++) {
    for (int col = 0; col < arr2D[0].length; col++) {
        total += arr2D[row][col];
    }
}

System.out.println(total); // Prints: 396
```